Gemini

If a cryptographer or prime-hunter hands a computer an arbitrary 150-digit integer, the computer initially has no idea how that number was constructed. To test it for the vulnerabilities we discussed, the code must **reverse-engineer** the number from the top down.

The Math Behind the "X-Ray"

We know that these generalized numbers are built using cyclotomic polynomials $\Phi_n(x)$. A fundamental rule of mathematics is that the highest exponent in any n-th cyclotomic polynomial is precisely defined by **Euler's Totient Function,** $\varphi(n)$.

Because of this, we don't have to guess the base. We can take an arbitrary number N and computationally extract its $\varphi(n)$-th root. If that root is a perfect integer, we have successfully cracked the hidden structural base.

Once the algorithm uncovers the underlying structure (n, the base b, and the exponent k), it checks for the two exact properties we discussed:

1. **The Treasure Map (Baseline Divisor):** It checks if k and n share no prime factors.
2. **The Skeleton Key (Aurifeuillian Trapdoor):** Instead of relying on complex, hardcoded mathematical rules, we can use a symbolic algebra engine to dynamically inject the geometric shape ($x = b \cdot y^2$) into the polynomial and command the computer to attempt an algebraic shatter.

The Python X-Ray Scanner

Because standard C is incapable of algebraic polynomial factorization, **Python's `sympy` (Symbolic Python)** library is the perfect tool for this test. It has built-in infinite precision math, integer root finders, and a symbolic algebraic engine.

Here is the complete script. You can run this in any Python environment (just run `pip install sympy` first).

Continue this chat

Python